Assignment 1 Part 1 – Amortized Analysis

Spring 2026

Due: Thursday, Feb 12 11:59 pm

To-do

Answer the following questions in detail. Your answer should be well structured and clearly explained. Feel free to cite references where applicable.

1. Use aggregate analysis to determine the amortized cost per operation for a sequence of n operations on a data structure in which the i^{th} operation costs i if i is an exact power of 2, and 1 otherwise.

Answer:

$$declare\ sum = 0$$

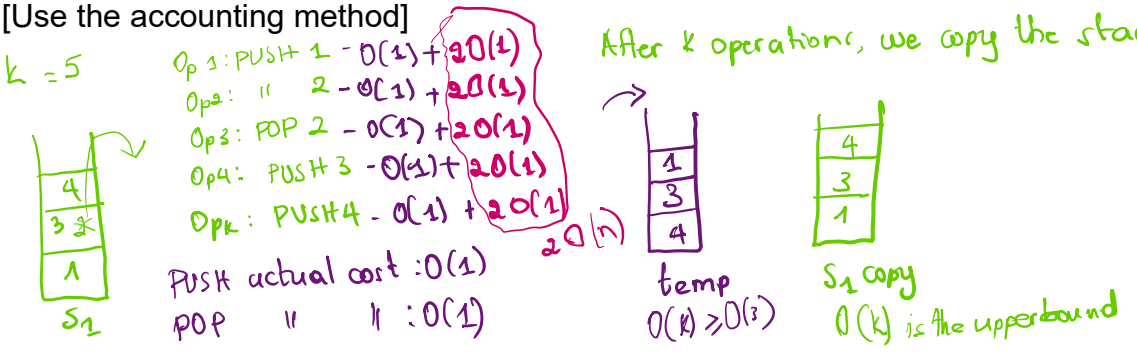
$$if\ i\ =\ 2^k for\ integer\ k, sum\ +=\ i$$

$$else\ sum+= 1$$

$$aggregate\_cost = \frac{sum}{n}$$

2. You perform a sequence of PUSH and POP operations on a stack whose size never exceeds k. After every k operations, a copy of the entire stack is made automatically, for backup purposes. Show that the cost of n stack operations, including copying the stack, is $O(n)$ by assigning suitable amortized costs to the various stack operations.

[Use the accounting method]

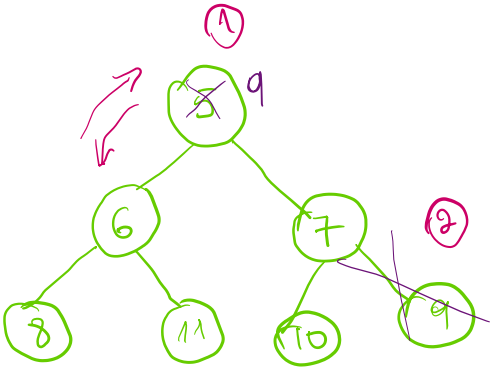


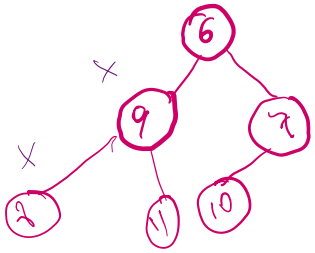
Answer: Using the accounting method, each PUSH or POP operation will be assigned an *amortized cost*, which "may be more or less than their actual cost" and will be used to pay for future expensive operations (Amortized Analysis slides). Since the size never exceeds k, the upper bound cost is $O(k)$ i.e. $n = k +$ cost of copying. If each PUSH and POP operation takes 3 times its actual cost, the $2 * O(n)$ becomes credit for the two operations that make up the copy operation, each costing $O(n)$.

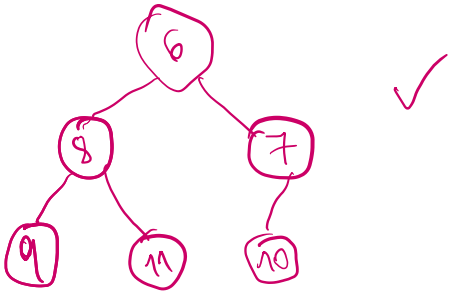
3. Consider an ordinary binary min-heap data structure supporting the instructions INSERT and EXTRACT-MIN that, when there are n items in the heap, implements each operation in $O(log\ n)$ worst-case time. Give a potential function φ such that the amortized cost of i NSERT is $O(log\ n)$ and the amortized cost of EXTRACT-MIN is $O(1)$ and show that your potential function yields these amortized time bounds. Note that in the analysis, n is the number of items currently in the heap, and you do not know a bound on the maximum number of items that can ever be stored in the heap.

Answer: In an ordinary binary min-heap, EXTRACT-MIN means the following according to GeeksforGeeks:

- Replace the root (or the element to be deleted) with the last element in the heap.
- Remove the last element from the heap, since it has been moved to the root.
- Heapify-down: The element now at the root may violate the Min-Heap property, so perform heapify starting from the root to restore the heap property


1
5
9
6
7
2
8
11
10
9


6
9
7
2
11
10


6
8
7
9
11
10

4. In a priority queue implemented using a binary heap, some operations (like insertions and deletions) might take longer when the heap needs to be restructured. Explain, in words, how amortized analysis is used to show that the average cost per operation (insert, delete) over a sequence of operations remains efficient, despite occasional restructuring of the heap.

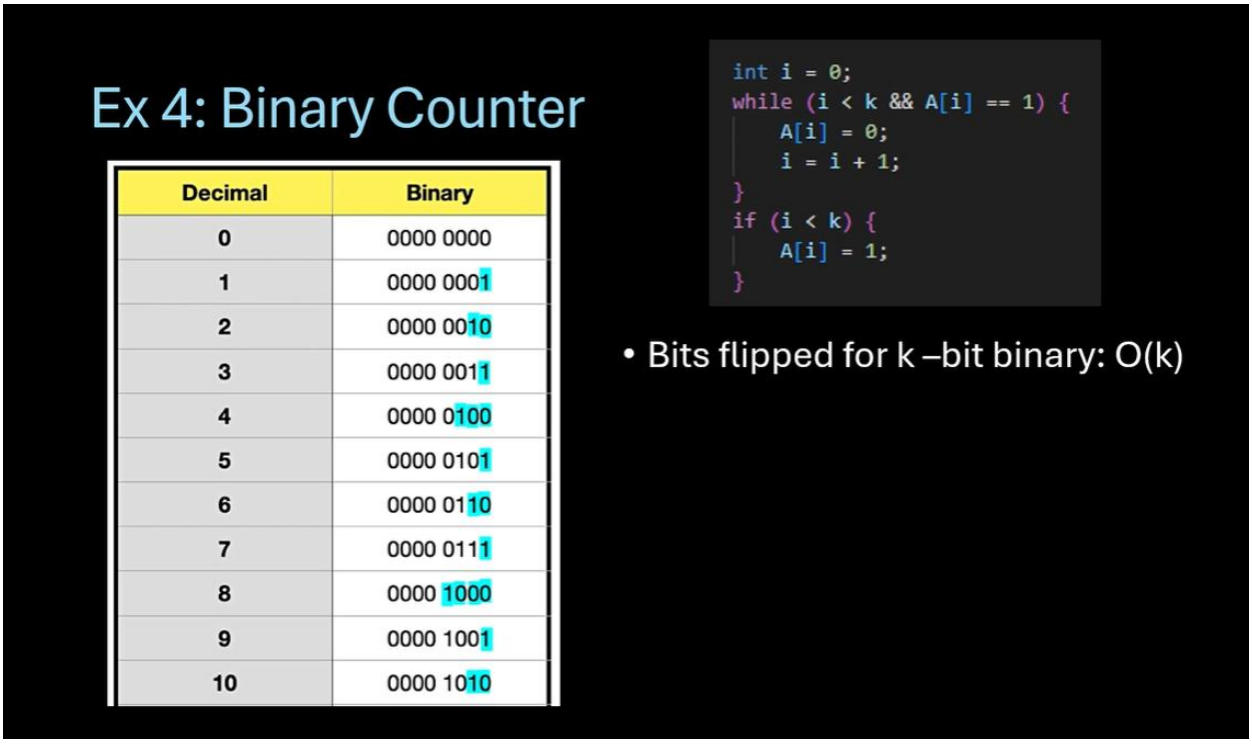
A priority queue implemented using a binary heap needs restructuring in these cases:

i. At insertion when the new node can only be a root of a subtree or the whole tree and we have to check if every node in that subtree maintains a binary heap structure. Otherwise, it finds a place at the next leftmost empty space.

ii. At deletion when we're removing the root of a subtree or the whole tree and need to heapify down to maintain a binary heap structure.

Both cases happen once in a while and do not capture a reasonable approximation of the priority queue's operational cost. Thus, amortized analysis explains how lightweight, frequent operations, like deleting without a need to heapify, can store credit/potential to pay for the occasional heavy operation.

5. Show the amortized analysis of Binary Counter implantation from our class slides using aggregate, accounting, and potential methods.



Aggregate method

Total cost of N operations: $\sum_{i=0}^{n} \frac{n}{2^n}$

Total cost for all N operations: $n = 8 \Rightarrow 8 + \frac{8}{2} + \frac{8}{4} + \frac{8}{8} + \frac{8}{16} + \frac{8}{32} + \frac{8}{64} + \frac{8}{128} =$

$= 8 + 4 + 2 + 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16}$

Half is 2^{i} the other 2^{-i}

Amortized cost:

Accounting method

Total actual cost:

Amortized cost per flip:

Credit accumulation:

Total amortized cost:

Potential method

Define the potential function: $\emptyset = number\ of\ i$ i.e. the number of bits in a string.

6. A splay tree is a self-adjusting binary search tree in which each operation (insert, delete, search) involves a process called "splaying," which moves the accessed node to the root. Describe how amortized analysis helps demonstrate that the amortized time complexity of a series of operations on a splay tree can be bounded by $O(logn)$, even though some individual operations may take longer. Show the proof of this for the ZagZig rotation.

Answer:

Accounting method

Total actual cost:

Amortized cost per flip:

Credit accumulation:

Total amortized cost:

Total Points (60)

- Each question is worth 10 points

Deliverables:

- A pdf file named A1_part_1 uploaded to D2L Dropbox and GitHub